

Selectors

Basic Selectors

special selectors for skipCondition

for action group skipCondition:

isPrevActionGroupSkipped returns TRUE if previous action group skipped

isPrevActionGroupDone returns TRUE if previous action group not skipped

for action skipCondition:

isPrevActionSkipped returns TRUE if previous action skipped

isPrevActionDone returns TRUE if previous action not skipped

selectors from modules

//'votes'

isTargetGotMajority({ voteResult, target }: { voteResult: IVoteResult; target: string }) => boolean

getVotersByAnswer({ voteResult, answer }: { voteResult: IVoteResult; answer: string }) => string[]

returns a list of players ids who voted for a particular player or poll option

example:

```
"selector": "getVotersByAnswer",
"params": [
  {
    "name": "voteResult",
    "type": "cached",
    "value": "lastActionResult"
  },
  {
    "name": "answer",
    "type": "preset",
    "value": "yes"
  }
]
```

getPlayerVoice({ voteResult, playerId }: { voteResult: IVoteResult; playerId: string }) => string[]

returns how a specific player voted. For any vote, the result is an array of strings, where the strings are player id's or targets.

//'teamsAndRoles'

getTeamNameByPlayerId({ playerId }: { playerId: string }) => string

getRoleNameByPlayerId({ playerId }: { playerId: string }) => string

getPlayersFromTeam({ teamId }: { teamId: string }) => string[]

getRoleName({ roleId }: { roleId: string }) => string

getPlayersByRole({ role }: { role: string }) => string[]

getPlayerByRole({ role }: { role: string }) => string

getPlayersRoles({ players }: { players: string[] }) => string[]

maps a list of player ids to their role ids, e.g. ["player_1", "player_2"] maps to ["townsperson", "minion"].

getPlayersTeams({ players }: { players: string[] }) => string[]

maps a list of player ids to their teams ids, e.g. ["player_1", "player_2"] maps to ["town", "mafia"].

getCardNames({ ids }: { ids?: string[] }) => string[]

getAllRolesIdsExpandedByFrequency() => string[]

returns a list of all the role ids that were put into the game, expanded by frequency

getTrueRoles() => TrueRoles

switchTrueRoles({ trueRoles, from, to }: { trueRoles: TrueRoles; from: string; to: string }) => TrueRoles

isTrueRoleInGame({ trueRoles, role }: { trueRoles: TrueRoles; role: string }) => boolean

getNonEmptyTeamsIds() => string[]

returns the ids of teams with a non-zero number of members.

getFewestPlayersTeam() => string

returns the id of (any) team with the fewest players.

getMostPlayersTeam() => string

returns the id of (any) team with the most players.

//playerScore'

getPlayersByScore({ min, max }: { min?: number, max?: number }) => string[]

returns all the players whose scores are less or equal than max (default: Infinity) but greater or equal than min (default: 0)

getPlayerScore({ playerId }: { playerId: string }) => number

getPlayersWithMinScore({ players }: { players?: string[] }) => string[]

if players are not defined, all players are taken.

getPlayersWithMaxScore({ players }: { players?: string[] }) => string[]

if players are not defined, all players are taken.

getMaxCurrentScore() => number

getMinCurrentScore() => number

//'deadPlayer'

allLivePlayers() => string[]

getAllPlayersExcept({ players, additionalPlayers }: { players: string[]; additionalPlayers?: string[] }) => string[]

//'cards'

fetchHandField({ playerId, field }: { playerId: string; field: string }) => string[]

returns a list of that field extracted from each card in the player's private hand

getCardField({ cardId, field }: { cardId: string; field: string }) => string

returns a field of a card

fetchDeckField({ deck, field }: { deck: string; field: string }) => array of card objects

getCardsIdsByType({ type }: { type: string }) => string[]

getPlayerByCardName({ name }: { name: string }) => string

getTrickWinner() => string

getCardsScore({ deck, hand }: { deck?: string; hand?: string }) => number (sum of points of all cards in the deck or hand)

playerHand({ playerId }: { playerId: string }) => array of card objects

getDeckCards({ deck }: { deck: string }) => array of card objects

getDeckLabel({ deck }: { deck: string }) => string

getNoThanksCardsScore({ deck, hand }: { deck?: string; hand?: string }) => number

getLlamaPartyCardsScore({ hand }: { hand: string }) => number

getHandCardsIdsByName({ playerId, name }: { playerId: string; name: string }) => string[]

getGalaxyBrainThinkerRoundScore(
 thinker,
 judgeChoices,
 thinkersChoices,
) : {
 thinker: string;
 judgeChoices: Record<string, string>;
 thinkersChoices: Record<string, string>;
} => number

getGalaxyBrainJudgeRoundScore(
 judgeChoices,
 thinkersChoices,
) : {
 judgeChoices: Record<string, string>;
 thinkersChoices: Record<string, string>;
} => number

getFugitivePlayableCards(
 playerId,
 highestCard,
 isFirstTurn,
) : {
 playerId: string;
 highestCard: number;
 isFirstTurn?: boolean;
} => string[]

```
//'grids'  
getCodenamesWinner() => string
```

selectors from applications

```
//'mafia'
```

```
livePlayersFromTeam({ teamId }: { teamId: string }) => string[]  
isAllTeamAlive({ teamName }: { teamName: string }) => boolean  
isNotRoleInGame({ role }: { role: string }) => boolean  
isVigilanteSkip({ prevChoice }: { prevChoice: IVoteResult['voteResult'] }) => boolean  
questFailsNumber({ voteResult, playerId }: { voteResult: IVoteResult; playerId: string }) => number
```

special selectors for creating a notification text (all these selectors return a string)

```
prepareVigilanteChoiceAnnounce({ ids }: { ids: string[] })  
prepareParityCopChoiceAnnounce({ currentChoice, prevChoice }: { currentChoice: string[]; prevChoice?:  
string[] })  
prepareAccuserNotification({ accuser, nominee }: { accuser: string[]; nominee: string })  
prepareDeathVoteQuestion({ player }: { player: string[] })  
prepareDeathVoteResultAnnounce({ player, voteId }: { player: string[]; voteId: string })  
prepareWerewolvesRevealAnnounce()  
prepareSeerChoiceAnnounce({ ids }: { ids: string[] })  
prepareEndGameAnnounce({ ids, trueRoles }: { ids: string[]; trueRoles: TrueRoles })
```

special selectors for winCondition (all these selectors return a boolean)

```
isWerewolvesWin({ ids, trueRoles })  
isTannerWins({ ids, trueRoles })  
isVillagersWins({ ids, trueRoles })
```